

程式語言

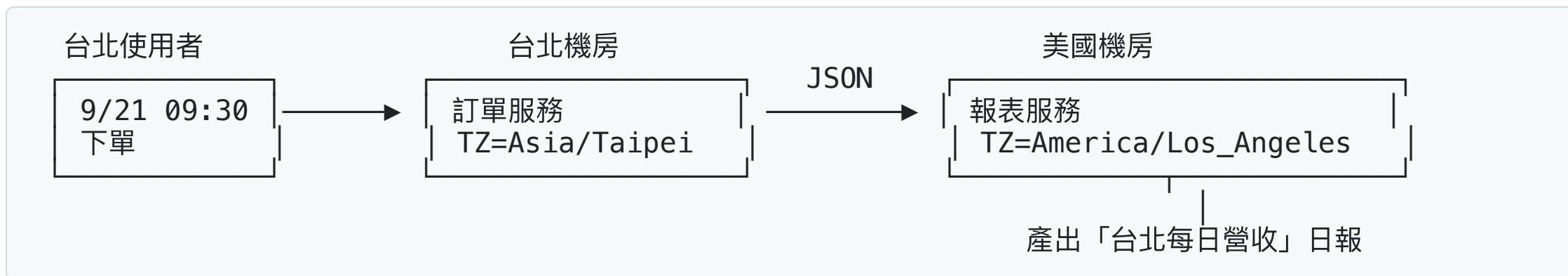
Programming Languages — Class 3

政大資碩 115-1 陳家宏

今天三堂課

時段	內容
第 1 堂	一筆訂單的時間 → §2.1 用介面指定資料 (pp. 31–34)
第 2 堂	§2.2 表示法策略：environment (pp. 35–42)
第 3 堂	兩個隨堂演練：Ex 2.4 stack 規格、Ex 2.5 a-list 實作

一個故事：一筆訂單的時間



訂單服務要把 `created-at` 放進 JSON 傳過去。

JSON 的值只有六種：string、number、object、array、true/false、null。

裡面沒有「時間」。所以時間一定得借用其中一種來表示。

做法一：整數 timestamp

```
{"order-id": 42, "created-at": 1789954200000}
```

1970-01-01T00:00Z 起算的毫秒數。這個數在台北和美國都指向同一個時間點：

```
(java.time.Instant/ofEpochMilli 1789954200000)  
;=> #object[java.time.Instant "2026-09-21T01:30:00Z"]
```

傳過去、讀回來，時間點正確。

代價：客服拿著 log 裡的 1789954200000 來問「這筆是幾點下的？」——
你得先開一個 REPL 才能回答。

做法二：字串

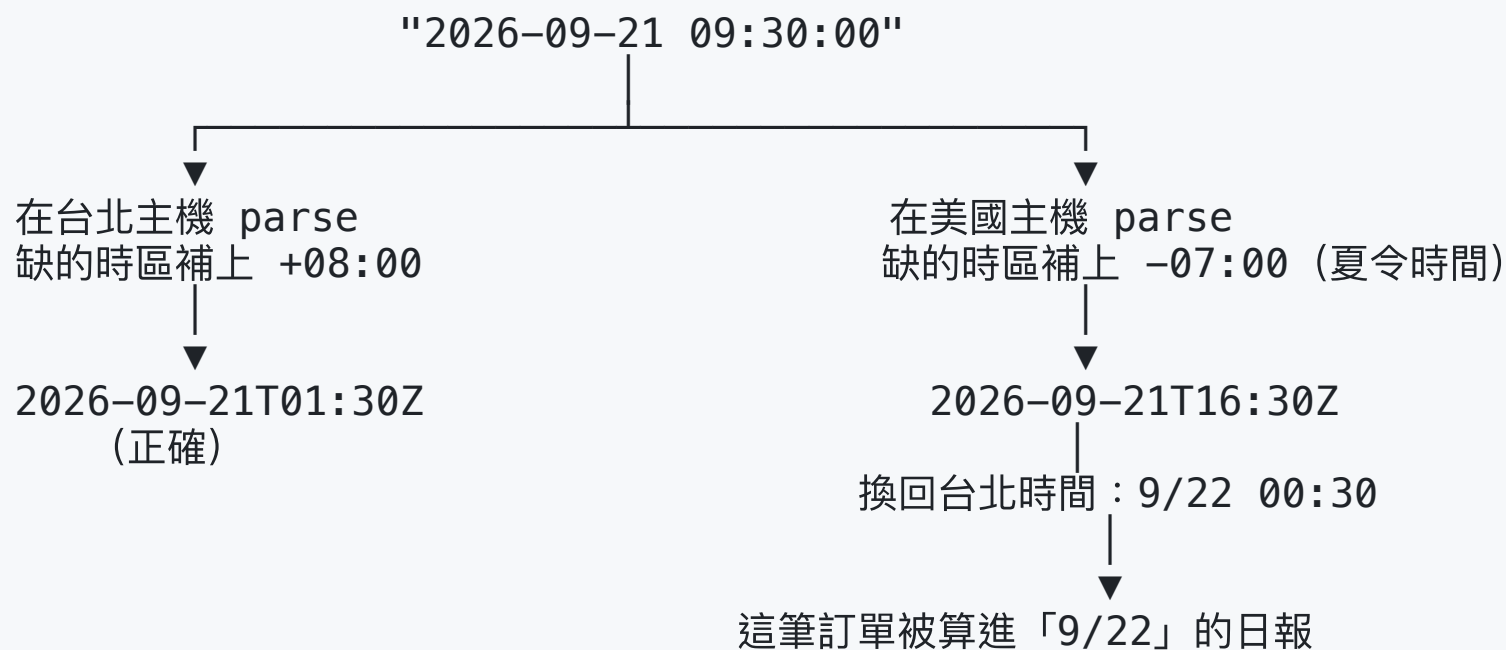
```
{"order-id": 42, "created-at": "2026-09-21 09:30:00"}
```

人讀得懂，log 裡一眼就知道是 9/21 早上九點半。

報表服務收到之後，把它 parse 成時間點，再依「台北日期」分組加總。

這筆訂單會被算進哪一天的日報？

同一個字串，兩個時間點



哪裡出錯了？

```
(defn parse-local [s zone]
  (-> (LocalDateTime/parse s (DateTimeFormatter/ofPattern "yyyy-MM-dd HH:mm:ss"))
      (.atZone (ZoneId/of zone))
      .toInstant))

(parse-local "2026-09-21 09:30:00" "Asia/Taipei")
;=> 2026-09-21T01:30:00Z

(parse-local "2026-09-21 09:30:00" "America/Los_Angeles")
;=> 2026-09-21T16:30:00Z
```

字串裡沒有時區。 `zone` 這個參數在實務上沒人傳——它由讀的那一方的主機設定補上。

同一個字串，`parse` 的結果取決於一個不在字串裡的東西。

修補：字串帶上 offset

```
{"order-id": 42, "created-at": "2026-09-21T09:30:00+08:00"}
```

人讀得懂，時間點也正確。

但現在每個服務、每個讀到這一欄的函式，都得知道三件事：

要知道的事	誰負責
寫出時要帶 offset	每一個寫出時間的地方
讀入時要用哪個 parser	每一個讀取時間的地方
哪些欄位是時間、哪些是普通字串	每一個碰到這筆資料的地方

parse 住在哪裡？

報表服務裡的兩個函式：

```
(defn daily-revenue [orders day]
  (->> orders
    (filter #(= day (-> (:created-at %)
                        OffsetDateTime/parse           ; knows the wire format
                        (.atZoneSameInstant taipei)
                        .toLocalDate)))

    (map :amount)
    (reduce +)))

(defn late-orders [orders deadline]
  (filter #(.isAfter (OffsetDateTime/parse (:created-at %)    ; knows it again
                                             deadline)
            orders))
```

兩個都在算營收、算逾期，但兩個都知道「created-at 是一個 ISO 8601 字串」。
哪天格式要換，得把所有這種地方找出來。

edn 的規格怎麼描述這件事

Users of data formats without such facilities must rely on either convention or context to convey elements not included in the base set. This greatly complicates application logic, betraying the apparent simplicity of the format.

— edn spec, Rationale (github.com/edn-format/edn)

原文	在我們的故事裡
data formats without such facilities	JSON
convention	「 <code>created-at</code> 用 ISO 8601、要帶 offset」這個約定
context	寫的那一方的主機時區
complicates application logic	上一張那兩個函式裡的 <code>OffsetDateTime/parse</code>

edn 的做法：#inst

```
;; on the wire
"{:order-id 42 :created-at #inst \"2026-09-21T09:30:00.000+08:00\"}"

(def order (clojure.edn/read-string msg))

(class (:created-at order))
;=> java.util.Date

order
;=> {:order-id 42, :created-at #inst "2026-09-21T01:30:00.000-00:00"}
```

`#inst` 是一個 **tag**：它告訴 reader「後面那個字串代表一個時間點」。reader 讀到 tag，先讀後面的字串，再交給這個 tag 的 handler 解讀，程式拿到的是 handler 產生的值（edn spec, tagged elements 節）。

parse 移到哪裡了？

JSON + 帶 offset 的字串

wire "2026-09-21T09:30:00+08:00"

↓
JSON parser → 字串

daily-revenue	(parse 字串)
late-orders	(parse 字串)
...	(parse 字串)

parse 散在各個函式裡

edn + #inst

#inst "2026-09-21T09:30:00.000+08:00"

↓
edn reader
#inst handler → 時間資料型態

daily-revenue
late-orders
...

← 牆

函式只看到時間資料型態

同一個時間點，兩種寫法

```
(= #inst "2026-09-21T09:30:00.000+08:00"  
   #inst "2026-09-21T01:30:00.000Z")  
;=> true
```

edn spec 的 equality 節：兩個 `#inst` 只要字串依 RFC-3339 指向同一個時間點，就相等。

時間點的介面長什麼樣？

令 t 為時間點在時間軸上的座標（毫秒）， $[t]$ 為它的表示：

```
;; constructors
(from-epoch-ms n)          = [n]
(plus-ms [t] d)            = [t + d]

;; observers
(epoch-ms [t])             = t
(before? [t1] [t2])        = #t  if t1 < t2
                           #f  otherwise
(local-date [t] zone)      = t 在 zone 的規則下對應的日曆日期
```

`late-orders` 只用到 `before?`；`daily-revenue` 只用到 `local-date`。

`epoch` 整數、帶 `offset` 的字串、`#inst`、`Date`，都是 $[t]$ 。

注意 `zone` 列在參數表上。故事裡的 bug，是 `zone` 從參數表上消失，改由主機設定補上。

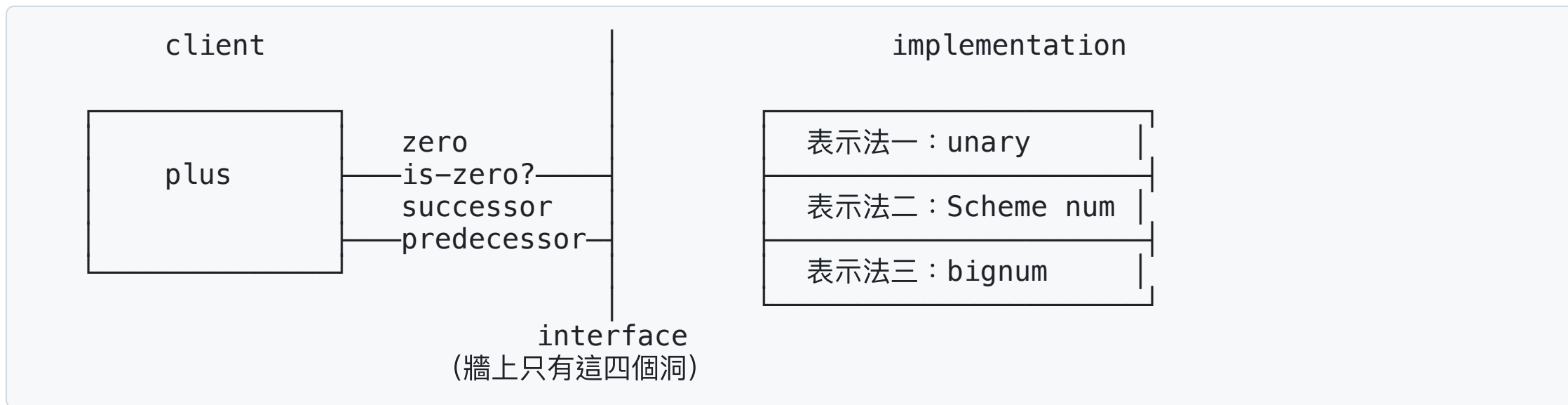
從故事到課本

故事裡	課本 §2.1 的說法
<code>daily-revenue</code> 、 <code>late-orders</code>	client
上一張的五個程序與等式	interface
epoch 整數、帶 offset 的字串、 <code>#inst</code> 、 <code>Date</code>	representation (表示法)
<code>OffsetDateTime/parse</code> 散在各函式裡	client 依賴了「表示法」

課本接下來用一個更小的例子——自然數——把這件事講到可以逐條檢查。

interface 與 implementation

課本 p.31 把資料型別切成兩半：



representation-independent

課本 p.32 :

當 client 只透過介面中的程序來操作某型別的值時，我們說這段 client 程式碼是 **representation-independent** 的——因為它不依賴該型別的值如何被表示。

注意這句話的主詞：具有這個性質的是 **client 程式碼**。

同一個資料型別，可以有 representation-independent 的 client，也可以有偷看表示法的 client。

[v] 這個記號

課本 p.32 引入一個記號，後面自然數與 environment 的等式都用它來寫：

$[v]$ = 「資料 v 的表示」

所以 [3] 指的是某個實作裡代表 3 的那個東西。

表示法	[3] 是什麼
unary	(#t #t #t)
Scheme number	3
bignum (N=16)	(3)

例子：自然數的介面

課本 p.32 挑了最小的例子——自然數。

介面由四個程序構成：

```
(zero)  
(is-zero? n)  
(successor n)  
(predecessor n)
```

課本接著說：當然，**並非隨便一組程序**都能當作這個介面的實作。
要被接受，必須滿足下一張的四條等式。

四條等式 (p.32)

$$(\text{zero}) = [0]$$

$$(\text{is-zero? } [n]) = \begin{cases} \text{\#t} & n = 0 \\ \text{\#f} & n \neq 0 \end{cases}$$

$$(\text{successor } [n]) = [n + 1] \quad (n \geq 0)$$

$$(\text{predecessor } [n + 1]) = [n] \quad (n \geq 0)$$

課本 p.32：這份規格**沒有規定**這些自然數要怎麼被表示，只要求這些程序合起來產生指定的行為。

(predecessor (zero)) 會怎樣？

看第四條等式：

$$(\text{predecessor } [n + 1]) = [n] \quad (n \geq 0)$$

左邊的形狀是 $[n+1]$ ，而 $n \geq 0$ ，所以它涵蓋 $[1]$ 、 $[2]$ 、 $[3]$
 $[0]$ 不在裡面。

課本 p.32 明講：規格對 (predecessor (zero)) 一句話也沒說，
所以在這份規格之下，任何行為都是可接受的。

待會三種實作各跑一次這行，看看會發生什麼。

client 程式：plus

```
(define plus
  (lambda (x y)
    (if (is-zero? x)
        y
        (successor (plus (predecessor x) y)))))
```

課本 p.33：不論我們採用哪一種自然數的實作，這支程式都會滿足

$$(\text{plus } [x] [y]) = [x + y]$$

這支程式碼裡沒有任何一行知道自然數長什麼樣子。

constructors 與 observers

課本 p.33 :

多數介面會包含一些 **constructor**（建造出該型別的元素）
與一些 **observer**（從該型別的值中取出資訊）。

自然數這個介面：

角色	成員
constructor	zero 、 successor 、 predecessor
observer	is-zero?

表示法一：unary

課本 p.33。用長度為 n 的 `#t` 串列表示 n ：

$$[0] = () \quad [n + 1] = (\text{\#t} . [n])$$

```
(define zero      (lambda () '()))  
(define is-zero?  (lambda (n) (null? n)))  
(define successor (lambda (n) (cons #t n)))  
(define predecessor (lambda (n) (cdr n)))
```

```
(plus '#t #t) '#t)    ; => (#t #t #t)  
(predecessor (zero))  ; => ?
```

表示法二：Scheme number

課本 p.33。直接用 Scheme 自己的整數，`[n]` 就是整數 n 。

```
(define zero      (lambda () 0))  
(define is-zero?  (lambda (n) (zero? n)))  
(define successor (lambda (n) (+ n 1)))  
(define predecessor (lambda (n) (- n 1)))
```

```
(plus 2 1)      ; => 3  
(predecessor (zero)) ; => ?
```

課本附註：Scheme 內部怎麼表示數字，本身可能也相當複雜。

表示法三：bignum

課本 p.34。以 N 為底，把數字表示成一串 0 到 $N-1$ 的數（稱為 *bigits*），最低位在前：

$$[n] = \begin{cases} () & n = 0 \\ (r . [q]) & n = qN + r, 0 \leq r < N \end{cases}$$

取 $N = 16$ ：

n	[n]	驗算
33	(1 2)	$1 \times 16^0 + 2 \times 16^1 = 1 + 32 = 33$
258	(2 0 1)	$2 \times 16^0 + 0 \times 16^1 + 1 \times 16^2 = 2 + 0 + 256 = 258$

課本 p.34：這種表示法容易表達遠大於一個機器字組的整數。

現場替換：回到那道牆

三份 implementation，輪流 load。 **plus** 的定義一個字都不改。

```
(plus [2] [1])  
  
;; unary      ' (#t #t) ' (#t)  => (#t #t #t)  
;; scheme number 2      1      => 3  
;; bignum (N=16) '(2)    '(1)    => (3)
```

三次的回傳值長得完全不同，但每一次都是 **[3]**。

```
(predecessor (zero))  
  
;; unary      => error: cdr: contract violation  
;; scheme number => -1  
;; bignum      => error: car: contract violation
```

opaque 還是 transparent ?

課本 p.34：上面三個實作都沒有強制資料抽象。

client 隨時可以偷看表示法，判斷它是 list 還是 Scheme 整數。

名稱	定義 (p.34)
opaque	表示法被隱藏，任何操作（含列印）都無法揭露它
transparent	反之

Scheme 沒有建立 opaque 型別的標準機制。

課本 p.34 的處置：定義介面，然後仰賴 client 的作者自律。

第 1 堂回顧

1. 時間的故事：parse 散在 application logic 裡，換表示法時就得把它們全部找出來
2. 課文的說法：client 只透過介面操作，稱為 representation-independent (p.32)
3. 介面說明：資料代表什麼、有哪些操作、操作有哪些性質 (p.31)；
自然數的例子用四條等式寫出這些性質 (p.32)。
介面成員分成 constructor 與 observer (p.33)
4. 自然數的三種表示法：unary、Scheme number、bignum——plus 一行不改
5. 表示法被隱藏的型別稱為 opaque；Scheme 沒有這個機制，靠 client 自律 (p.34)

參考資料

- edn 規格：<https://github.com/edn-format/edn>

第 2 堂

§2.2 表示法策略：environment

§2.2 要做什麼？

- 第 1 堂談了 representation independence。這一節以它為前提，用 environment 當例子，看同一個介面可以有哪幾種表示法 (p.35)
- environment 用來把「變數」關聯到「值」
- 變數可以用任何我們喜歡的方式表示，只要能檢查兩個變數是否相等 (p.35)
(表示法任選；介面只要求一個操作：判斷相等)
- 課文選用的變數表示法是 Scheme symbol；其他選項如 string、數字

environment 是一個有限函數

課本 p.36 :

| environment 是一個函數，定義域是**有限**的變數集合，值域是所有 Scheme 值。

依照「有限函數 = 有限個有序對」的慣例，要表示的是所有形如

$$\{(var_1, val_1), \dots, (var_n, val_n)\}$$

的集合，其中 var_i 兩兩相異。

課本把變數 `var` 在 `env` 裡的值稱為它在 `env` 中的 **binding**。

三條等式 (p.36)

$$(\text{empty-env}) = [\emptyset]$$

$$(\text{apply-env } [f] \text{ var}) = f(\text{var})$$

$$(\text{extend-env } \text{var } v [f]) = [g], \quad g(\text{var}_1) = \begin{cases} v & \text{var}_1 = \text{var} \\ f(\text{var}_1) & \text{otherwise} \end{cases}$$

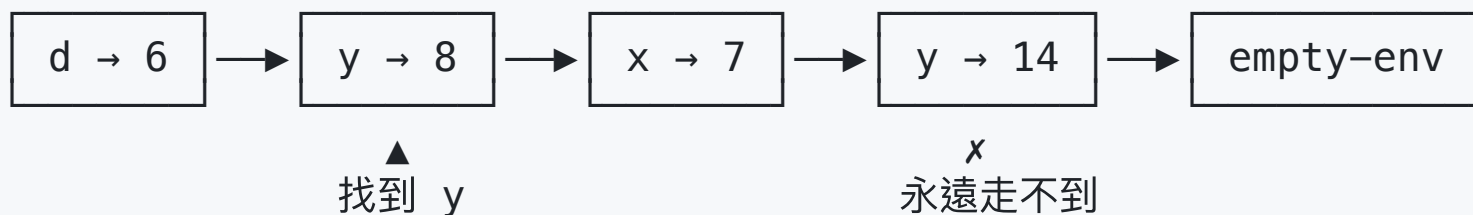
第三條的 g 是一個新的函數：除了 `var` 這一點以外，它跟 f 一模一樣。

`extend-env` 沒有修改 f 。

這個 e 的 y 是 8 還是 14？

```
(define e
  (extend-env 'd 6
    (extend-env 'y 8
      (extend-env 'x 7
        (extend-env 'y 14
          (empty-env)))))))
```

apply-env 的搜尋路徑（由外往內）



課本 p.36： $e(d) = 6$ 、 $e(x) = 7$ 、 $e(y) = 8$ ，其餘變數上未定義。

y 綁到 14 的那一次，被後來綁到 8 的那一次遮蔽了。

誰是 constructor ， 誰是 observer

課本 p.36 最後一段：

角色	成員
constructor	<code>empty-env</code> 、 <code>extend-env</code>
observer	<code>apply-env</code>

`apply-env` 是這個介面裡僅有的 observer 。

每個 environment 都長得出來嗎？

課本 p.37：每個 environment 都可以從 empty environment 出發，套用 `extend-env` n 次 ($n \geq 0$) 得到：

```
(extend-env varn valn
  ...
  (extend-env var1 val1
    (empty-env)))...
```

於是所有 environment 都是下面這個文法產生的句子：

```
Env-exp ::= (empty-env)
         ::= (extend-env Identifier Scheme-value Env-exp)
```

把「這個 `e` 的 `y` 是 8 還是 14？」那張的 `e` 寫成這個文法的一個句子。兩條 production，各用了幾次？

Figure 2.1 (上)：兩個 constructor

課本 p.38。用同一個文法來描述一組 list：

```
;; Env = (empty-env) | (extend-env Var SchemeVal Env)
;; Var = Sym

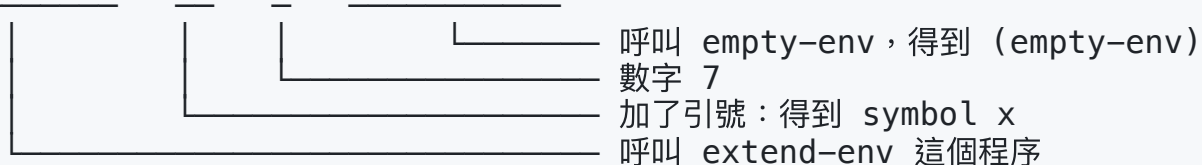
;; empty-env : () -> Env
(define empty-env
  (lambda () (list 'empty-env)))

;; extend-env : Var * SchemeVal * Env -> Env
(define extend-env
  (lambda (var val env)
    (list 'extend-env var val env)))
```

哪一個是呼叫，哪一個是資料？

程式碼：function call

`(extend-env 'x 7 (empty-env))`



↓ 求值

值：data (四個元素的 list)

<code>extend-env</code>	<code>x</code>	<code>7</code>	<code>(empty-env)</code>
-------------------------	----------------	----------------	--------------------------

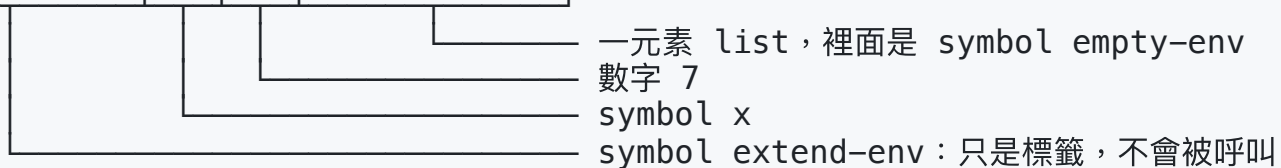


Figure 2.1 (下) : `apply-env`

```
;; apply-env : Env * Var -> SchemeVal
(define apply-env
  (lambda (env search-var)
    (cond
      ((eqv? (car env) 'empty-env)
       (report-no-binding-found search-var))
      ((eqv? (car env) 'extend-env)
       (let ((saved-var (cadr env))
             (saved-val (caddr env))
             (saved-env (cadddr env)))
         (if (eqv? search-var saved-var)
             saved-val
             (apply-env saved-env search-var))))
      (else
       (report-invalid-env env)))))
```

兩個錯誤回報程序

```
(define report-no-binding-found
  (lambda (search-var)
    (eopl:error 'apply-env "No binding for ~s" search-var)))

(define report-invalid-env
  (lambda (env)
    (eopl:error 'apply-env "Bad environment: ~s" env)))
```

兩者的觸發條件不同：

程序	什麼時候被呼叫
report-no-binding-found	輸入是合法的 env，但查不到這個變數
report-invalid-env	輸入根本不是這個表示法產生的東西

The Interpreter Recipe (p.37)

課本說 `apply-env` 是一個很常見的程式碼模式，並替它命名：

The Interpreter Recipe

1. 看一筆資料。
2. 判斷它代表哪一種資料。
3. 取出這筆資料的各個組成部分，依它的種類做該做的處理。

這是課本第五個加框命名的方法（前四個：Smaller-Subproblem Principle、Proof by Structural Induction、Follow the Grammar、No Mysterious Auxiliaries）。

分支數對得上 production 數嗎？

```
Env-exp ::= (empty-env)
         ::= (extend-env Identifier Scheme-value Env-exp)
```

```
(cond
  ((eqv? (car env) 'empty-env) ...) ; ← production 1
  ((eqv? (car env) 'extend-env) ...) ; ← production 2
  (else (report-invalid-env env))) ; ← ?
```

- 前兩個分支一一對應兩條 production（我的解讀：這是 class-02 的 Follow the Grammar，p.22）
- 第三個分支對應文法之外的輸入

一個 observer 是什麼樣的條件？

課本 p.40 §2.2.3 的第一句：

environment 的介面有一個重要性質：它恰好只有一個 **observer**，`apply-env`。
這讓我們可以把 environment 表示成一個 Scheme 程序——
它接收一個變數，回傳該變數的值。

作法：讓 `empty-env` 和 `extend-env` 回傳程序，
而那些程序被呼叫時，做的事情就是上一節 `apply-env` 做的事。

procedural representation (p.40)

```
;; Env = Var -> SchemeVal

;; empty-env : () -> Env
(define empty-env
  (lambda ()
    (lambda (search-var)
      (report-no-binding-found search-var))))

;; extend-env : Var * SchemeVal * Env -> Env
(define extend-env
  (lambda (saved-var saved-val saved-env)
    (lambda (search-var)
      (if (eqv? search-var saved-var)
          saved-val
          (apply-env saved-env search-var)))))

;; apply-env : Env * Var -> SchemeVal
(define apply-env
  (lambda (env search-var) (env search-var)))
```

同一個運算式，兩種長相

```
(extend-env 'x 7 (empty-env))
```

§2.2.2 資料結構表示法

```
(extend-env x 7  
  (empty-env))
```

印得出來
比得出相等
存得進檔案

§2.2.3 程序表示法

```
#<procedure>
```

裡面有 saved-var = x
saved-val = 7
saved-env = #<proc>

印出來只有 #<procedure>

兩邊滿足同一組等式（p.36 那三條），`apply-env` 的回傳值逐一相同。

兩種表示法對照

	§2.2.2 資料結構	§2.2.3 程序
Env 的定義	(empty-env) (extend-env ...)	Var -> SchemeVal
印得出內容	是	否
加一個 observer	改 apply-env 旁邊再加一個函式	見 Ex 2.13 ★★

最後一列是我的整理，課本沒有並列比較這兩者。

從 client 抽出介面的兩步 (p.41)

課本說「只有一個 observer」的情況比想像中常見：若要表示的資料本身就是函數，就可以用「把它套用到引數上會得到什麼」來表示它。

(environment 就是一例：它本身是函數，唯一的 observer `apply-env` 做的就是套用它。)

1. 找出 client 程式碼中，求值後會產生該型別的值的那些 lambda 運算式。
每一個都做成一個 constructor 程序，其參數就是該 lambda 的**自由變數**。
然後把 client 裡的那些 lambda 換成對應 constructor 的呼叫。
2. 定義一個 `apply-` 程序。找出 client (含 constructor 的 body) 裡所有「套用該型別的值」的地方，換成呼叫 `apply-`。

做完之後，介面 = 所有 constructor + 那個 `apply-`。

defunctionalization (p.41)

如果實作語言不支援高階程序，可以再走一步：

用資料結構表示法 + interpreter recipe 把上面那個介面實作出來。

課本 p.41 稱這個過程為 **defunctionalization**，並舉例：

推導出 environment 的資料結構表示法 (§2.2.2)，就是 defunctionalization 的一個簡單例子。

課本 p.41 說，程序表示法與 defunctionalized 表示法之間的關係，會在書中反覆出現。

第 2 堂回顧

1. environment 的介面：兩個 constructor、一個 observer，三條等式
2. 每個 env 都由 `empty-env` + n 次 `extend-env` 造出 $\Rightarrow$ `Env-exp` 文法
3. **The Interpreter Recipe**：看資料 $\rightarrow$ 判斷種類 $\rightarrow$ 取出組成部分，做該做的處理
4. 只有一個 observer $\Rightarrow$ 可以用程序來表示這個型別
5. 程序表示法 $\rightarrow$ 資料結構表示法，叫 defunctionalization

第 3 堂

寫一份規格，換一次實作

今天怎麼進行

時間	做什麼
0-22 分	演練一：Exercise 2.4 stack 規格（分組討論，各組發表）
22-45 分	演練二：Exercise 2.5 a-list 實作（分組討論，抽一組發表，跑測試）
45-50 分	下週預告

演練一：Exercise 2.4 ★★ (p.37)

考慮「值的 stack」這個資料型別，其介面由 `empty-stack`、`push`、`pop`、`top`、`empty-stack?` 這些程序構成。請依照前面例子的風格，為這些操作寫出規格。

哪些是 constructor，哪些是 observer？

「前面例子的風格」指的是 p.32 的四條自然數等式與 p.36 的三條 environment 等式。去翻那兩頁，照著寫。

寫之前先問一句

判斷 constructor / observer 的一個作法：看回傳型別。

回傳	分類
一個 stack	constructor
一個 stack 以外的東西	observer

用這個判準檢查一下：

pop 回傳什麼？ top 回傳什麼？兩者能不能合併成一個操作？

(課本 p.33 的說法是 constructor 「建造該型別的元素」、observer 「從該型別的值中取出資訊」。)

stack 規格的骨架

把等號右邊填滿。用 [s] 表示 stack s 的表示。

(empty-stack) = ?

(empty-stack? (empty-stack)) = ?

(empty-stack? (push v [s])) = ?

(top (push v [s])) = ?

(pop (push v [s])) = ?

(top (empty-stack)) = ?

(pop (empty-stack)) = ?

← 先想清楚這一格該不該填

←

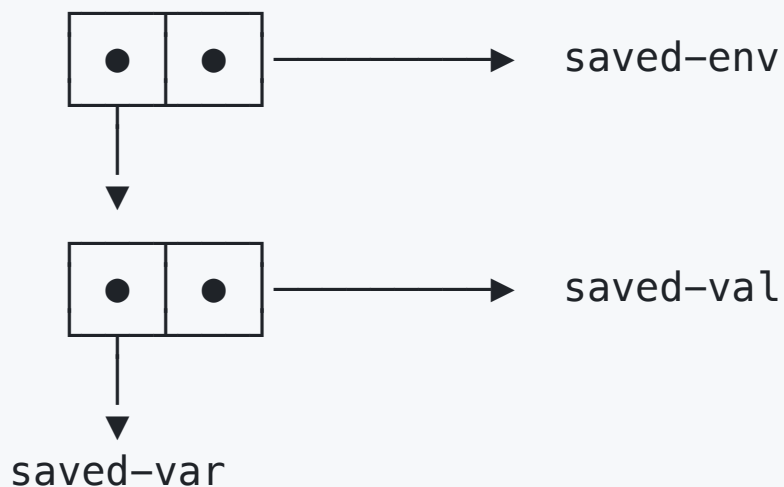
分組討論

1. 分成 3 組，自由討論。
2. 寫 stack type 的：(a) 操作規格。(b) 哪些是 constructor，哪些是 observer？
3. 10 minutes 後，各組上台發表。

演練二：Exercise 2.5 ★ (p.39)

只要能區分空的與非空的 environment，且能從非空的取出各部分，任何資料結構都可以拿來表示 environment。

課本 p.39 的圖（重繪）：空 environment 用 `'()`，`extend-env` 造出——



也就是 `((var . val) . saved-env)`。

分組討論

1. 分成 3 組，自由討論。
2. 各組設法寫出 a-list 表示法的 environment 實作
3. 10 minutes 後，抽一組上台發表。

演練二的測試

三份實作輪流 load，跑同一組測試。各組寫的 a-list 版本，也要跑出 (6 7 8)。

```
(define e
  (extend-env 'd 6
    (extend-env 'y 8
      (extend-env 'x 7
        (extend-env 'y 14 (empty-env))))))

(list (apply-env e 'd) (apply-env e 'x) (apply-env e 'y))
;; §2.2.2 資料結構 => (6 7 8)
;; §2.2.3 程序      => (6 7 8)
;; Ex 2.5 a-list    => (6 7 8)

e
;; §2.2.2 => (extend-env d 6 (extend-env y 8 ...))
;; §2.2.3 => #<procedure>
;; Ex 2.5 => ((d . 6) (y . 8) (x . 7) (y . 14))
```

下週預告

week 4 : §2.3–2.5

- 遞迴資料型別的介面 (constructor / predicate / extractor)
- `define-datatype` 與 `cases`
- concrete syntax 與 abstract syntax 的區別、`parse` / `unparse`
- HW2 發布

本週請完成：

- ☐ HW1 繳交 (week 4 上課前)
- ☐ §2.1–2.2 讀過一遍，Ex 2.4、2.5 確定自己會寫。

本日結論

介面規定有哪些操作，以及它們必須滿足的性質。

實作（表示法 + 操作的程式碼）是滿足這些性質的其中一種做法。

今天驗證過的：`plus` 換三種自然數表示法、`apply-env` 換三種 environment 表示法，client 的程式碼完全不需要改動。

問題時間

1. Email: laurence@replware.dev
2. Office Hour: 週四下午 1:00~2:00，請先跟我約
3. 助教：邵振皓 (負責改作業、監考)
4. 助教信箱：114971020@nccu.edu.tw
5. **重要修改**：改用 Moodle 來收作業